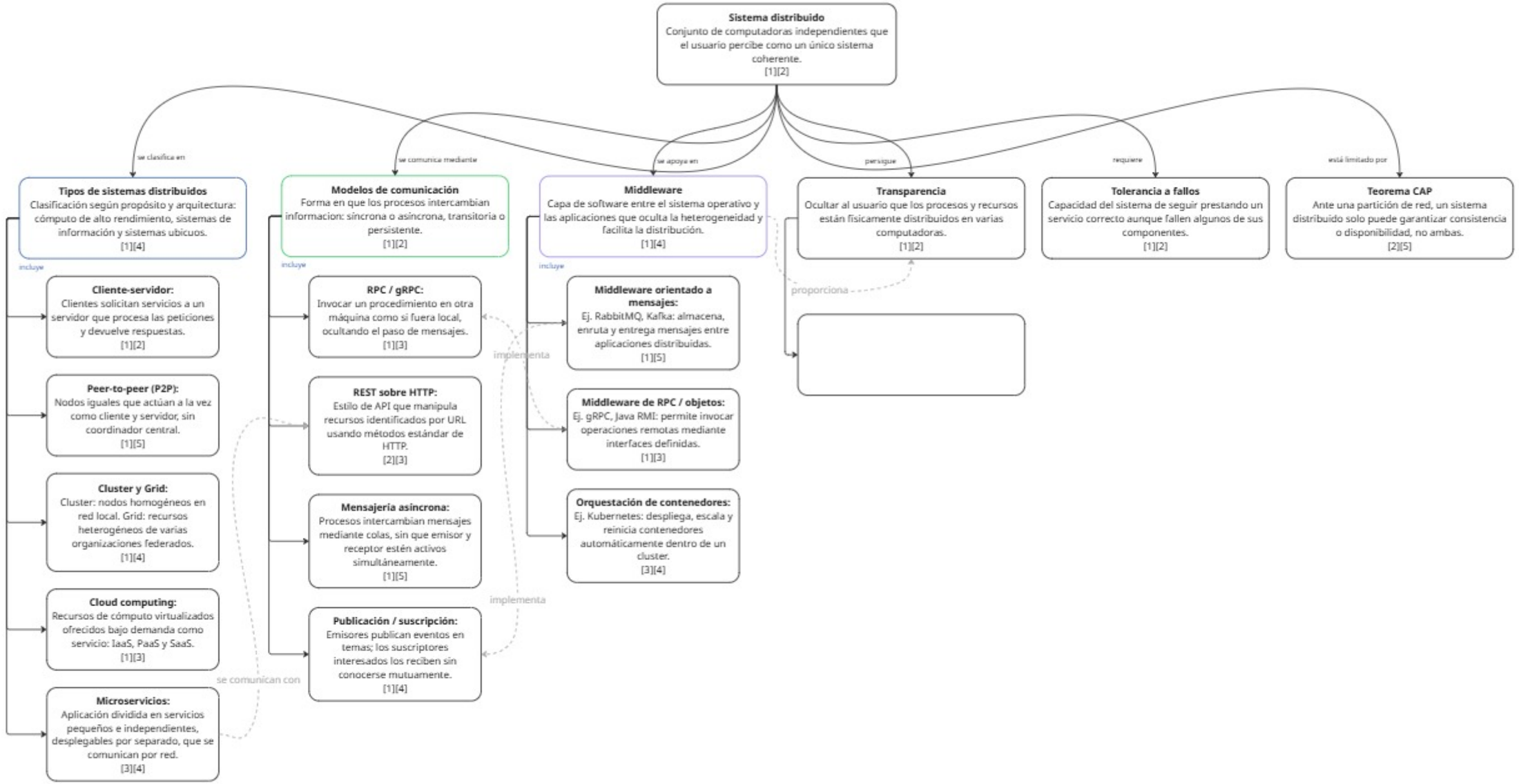

Sistema distribuido
Conjunto de computadoras independientes que el usuario percibe como un único sistema coherente.
[1][2]
se clasifica en
se comunica mediante
se apoya en
persigue
requiere
está limitado por
Tipos de sistemas distribuidos
Clasificación según propósito y arquitectura: cómputo de alto rendimiento, sistemas de información y sistemas ubicuos.
[1][4]
incluye
Cliente-servidor:
Clientes solicitan servicios a un servidor que procesa las peticiones y devuelve respuestas.
[1][2]
Peer-to-peer (P2P):
Nodos iguales que actúan a la vez como cliente y servidor, sin coordinador central.
[1][5]
Cluster y Grid:
Cluster: nodos homogéneos en red local. Grid: recursos heterogéneos de varias organizaciones federados.
[1][4]
Cloud computing:
Recursos de cómputo virtualizados ofrecidos bajo demanda como servicio: IaaS, PaaS y SaaS.
[1][3]
Microservicios:
Aplicación dividida en servicios pequeños e independientes, desplegables por separado, que se comunican por red.
[3][4]
Modelos de comunicación
Forma en que los procesos intercambian información: síncrona o asíncrona, transitoria o persistente.
[1][2]
incluye
RPC / gRPC:
Invocar un procedimiento en otra máquina como si fuera local, ocultando el paso de mensajes.
[1][3]
REST sobre HTTP:
Estilo de API que manipula recursos identificados por URL usando métodos estándar de HTTP.
[2][3]
Mensajería asíncrona:
Procesos intercambian mensajes mediante colas, sin que emisor y receptor estén activos simultáneamente.
[1][5]
Publicación / suscripción:
Emisores publican eventos en temas; los suscriptores interesados los reciben sin conocerse mutuamente.
[1][4]
se comunican con
Middleware
Capa de software entre el sistema operativo y las aplicaciones que oculta la heterogeneidad y facilita la distribución.
[1][4]
incluye
Middleware orientado a mensajes:
Ej. RabbitMQ, Kafka: almacena, enruta y entrega mensajes entre aplicaciones distribuidas.
[1][5]
Middleware de RPC / objetos:
Ej. gRPC, Java RMI: permite invocar operaciones remotas mediante interfaces definidas.
[1][3]
Orquestación de contenedores:
Ej. Kubernetes: despliega, escala y reinicia contenedores automáticamente dentro de un cluster.
[3][4]
implementa
implementa
Transparencia
Ocultar al usuario que los procesos y recursos están físicamente distribuidos en varias computadoras.
[1][2]
proporciona
Tolerancia a fallos
Capacidad del sistema de seguir prestando un servicio correcto aunque fallen algunos de sus componentes.
[1][2]
Teorema CAP
Ante una partición de red, un sistema distribuido solo puede garantizar consistencia o disponibilidad, no ambas.
[2][5]